

Nombres, alcance y ligaduras

Material de lectura sobre el tema: Complementar lo presentado aquí con:

- Sebesta: Capítulo 5.
- Scott: Capítulo 3. Recuerden manejar el nivel de profundidad de la lectura, Scott hace las explicaciones incluyendo detalles de implementación.

La carrera de la abstracción

Desde los primeros lenguajes, la evolución siempre ha estado marcada por el grado de abstracción. El primer paso en esta carrera estuvo dado por la incorporación de los **nombres**. Este detalle les brinda a los lenguajes la primera oportunidad para diferenciarse, y da lugar a varias preguntas de diseño detrás de esta característica. Las respuestas dan un abanico interesante de diferencias.

Los **nombres** o **identificadores** son básicamente una cadena de caracteres que se utilizan para identificar entidades en un programa. El conjunto de caracteres permitido siempre es un subconjunto del alfabeto a partir del cual se define el lenguaje. Las consideraciones de diseño que se deben tener en cuenta son las siguientes:

- ✧ ¿Tienen longitud máxima?
- ✧ ¿El lenguaje es sensible a mayúsculas y minúsculas?
- ✧ ¿Se permite el uso de conectores?
- ✧ ¿Hay palabras especiales (claves o reservadas)?

Las respuestas a estas preguntas tienen sus implicancias sobre la evaluación del lenguaje.

En el pasado, la **longitud máxima** de los nombres era una cuestión importante porque el ahorro de memoria era importante. Los lenguajes modernos, en general, no tienen restricciones en el tamaño o longitud de los identificadores. En el intermedio, algunos lenguajes tenían una postura híbrida, le dejaban creer al programador que no había restricciones de longitud, pero internamente podaban los identificadores a una determinada cantidad de caracteres. Este detalle de implementación presentó problemas para algunos lenguajes, ya que de no hacerse con cuidado podía provocar que más de un nombre colisionara en el mismo nombre podado, y por lo tanto se perdía la identidad de cada uno y todos internamente hacían referencia a lo mismo.

La **sensibilidad a mayúsculas y minúsculas** es un tema debatible. En un principio uno creería que cuenta con una mayor cantidad de nombres, pero el mal uso de esta característica rápidamente genera problemas de legibilidad. Por ello, la mayoría de los lenguajes que adoptaron esta facilidad terminan proponiendo convenciones para los nombres. Para ilustrar la situación ¿qué tan fácil sería leer e identificar errores en un programa con dos variables de tipos diferentes, pero con los nombres “*Variable*” y “*variable*”?

El uso de **conectores** no trae demasiadas implicancias, más allá de que los conectores deben estar en el alfabeto y deben ser tales que no generen confusión con las expresiones que se escriban en el lenguaje (por ejemplo, el lenguaje no debería permitir usar guiones medios para los nombres en caso de que pueda generar confusión con la operación de resta).

Las **palabras especiales** tienen diferentes maneras de ser introducidas en el lenguaje:

- ✧ **Palabras clave:** funcionan como separadores. Una palabra clave es un identificador que solo es especial (en interpretación) en ciertos contextos. Por ejemplo, en Fortran: en la sentencia “Real VarName” Real es una palabra clave en este contexto mientras que en esta otra sentencia “Real = 3.4” el identificador Real es aquí el nombre de una variable. Al ser especial solo dependiendo del contexto, la misma palabra puede coexistir con diferentes significados en el mismo momento.
- ✧ **Palabras reservadas:** es una palabra clave que no puede redefinirse por el usuario. Tienen su semántica claramente definida en el lenguaje y no es posible modificarla.
- ✧ **Palabras predefinidas:** conjunto de identificadores con un significado predeterminado, pero que puede ser modificado por el programador. A diferencia de las palabras clave, las predefinidas solo pueden aparecer con un significado en cada momento.

El abuso de las palabras especiales en un lenguaje perjudica su aprendizaje y evolución... ¿Podés pensar en los motivos?

Los atributos de los nombres

Cada nombre, dentro del lenguaje de programación, está definido a través de un conjunto de atributos (noten que el nombre por sí solo no alcanza, ya que en general puede haber más de una entidad con el mismo nombre). Cuáles son esos atributos depende de a qué entidad está asociado ese nombre.

La primera entidad que incorporaron los lenguajes de programación fueron las **variables**. Para las variables, el conjunto de atributos está dado por:

- ✧ **Nombre:** aunque no todas las variables poseen uno (hay variables conocidas como *variables anónimas*).
- ✧ **Dirección:** la dirección de memoria a la que está asociada. Una misma variable puede estar asociada a locaciones de memoria diferentes durante la ejecución del programa, y en diferentes lugares del programa. *Es importante notar que este atributo está en el conjunto en aquellos lenguajes que pertenecen a paradigmas convencionales. ¿Por qué?*
 - **Aliasing:** dos nombres de variables diferentes pueden utilizarse para acceder a la misma locación de memoria al mismo tiempo. El aliasing perjudica la legibilidad de los programas, ya que el programador que lee el programa debe tener presente todos los alias de una locación. Los alias se pueden crear a través del uso de punteros, variables por referencia. *Es importante tener en cuenta que para que el aliasing tenga lugar, no alcanza con la condición de que dos nombres estén asociados a la misma locación o dirección de memoria, sino que además ambos nombres deben estar disponibles para su uso. Esta última condición apela a que los cambios en la locación se pueden hacer indistintamente con ambos nombres. ¿Se te ocurre un ejemplo en el que esto ocurre y uno en el que no?*
El aliasing es un concepto que viene asociado a la interpretación de la variable como abstracción de lo almacenado en una celda de memoria.
- ✧ **Valor.**
- ✧ **Tipo:** determina el rango de valores para la variable y las operaciones que están definidas para los valores de ese tipo.
- ✧ **Tiempo de vida:** El tiempo de vida es el período de tiempo entre la creación y destrucción de la ligadura entre un nombre y un objeto. El tiempo de vida del objeto corresponde al mecanismo de asignación de memoria.
- ✧ **Alcance:** es la región textual de un programa en el cual una ligadura está activa. Una variable tiene una ligadura activa para una sentencia si puede ser referenciada en ella.

Hay un concepto que resulta fundamental en este contexto, el concepto de **ligadura**. Una ligadura es la asociación entre dos cosas. Resulta muy importante aquí porque cada atributo de una variable tiene una ligadura, esta ligadura ocurre entre el atributo de la variable y el valor del mismo.

Es importante tener en cuenta el momento en el cual se crea una ligadura, que llamamos el **tiempo de ligadura**. Los tiempos de ligadura están asociados al momento en el cual se toman decisiones vinculadas al mismo. Estos tiempos pueden ser estáticos o dinámicos. Los momentos donde se puede producir una ligadura son:

- ✧ **Diseño del lenguaje:** muchos de los aspectos de semántica se establecen en este momento (por ejemplo, el flujo de control, conjunto de tipos).
- ✧ **Implementación del lenguaje:** la mayoría de los lenguajes dejan detalles a la discreción del implementador (el número de bits para la representación de un tipo, tamaño del heap y pila, manejo de excepciones en tiempo de ejecución).
- ✧ **Escritura del programa:** algoritmos, estructuras de datos, nombres.
- ✧ **Compilación:** mapeo de los constructores de alto nivel a código máquina o datos estáticos a memoria.
- ✧ **Linkeo:** la compilación separada hace que algunas ligaduras no puedan resolverse en compilación.
- ✧ **Carga:** hace referencia al momento en el que el sistema operativo carga en memoria el programa. En el linkeo se establecen direcciones virtuales, en la carga se resuelven las direcciones físicas.
- ✧ **Ejecución:** cubre las ligaduras que tienen lugar desde el comienzo al fin de la ejecución, por ejemplo, el valor de las variables.

Para que la ligadura se considere dinámica debe ocurrir en tiempo de ejecución, mientras que cualquiera de los otros momentos están asociados a lo que se define como ligadura estática. No todos los lenguajes tendrán ligaduras en todos los momentos, en particular algunos de estos momentos no están disponibles para algunos lenguajes debido a la naturaleza de su traductor.

La ligadura temprana está asociada a mayor eficiencia, mientras que la más demorada a mayor flexibilidad.

El tiempo de ligadura de cada atributo de las variables brinda todo un abanico de posibilidades para los lenguajes. Por ejemplo, si se tiene la posibilidad de tener una ligadura estática para el valor, el lenguaje estaría brindando la posibilidad de definir constantes nombradas. Claramente no tendría sentido que todas las variables tengan la ligadura al valor de manera estática ya que no contaría con variables. Recuerden que la idea de la ligadura estática es que una vez que se creó el vínculo entre el atributo y el valor del atributo, ese vínculo se mantiene inalterado durante todo su tiempo de vida.

Si pensamos en la ligadura del tipo, es importante notar que esta ligadura debe estar resuelta antes de que una variable pueda ser utilizada. Es relevante entonces decidir dos aspectos: cómo se especifica el tipo, y cuándo la ligadura toma lugar.

Si la ligadura es **estática**, el tipo se determina a través de una declaración explícita o implícita (la declaración implícita es un mecanismo por default para especificar el tipo de una variable, y se aplica sobre la primera aparición de la variable en el programa. Un ejemplo sería incorporar en el lenguaje algún tipo de regla como “todo identificador que comience con la letra R si no fue declarado explícitamente de otro tipo queda declarado de tipo real”). Este tipo de ligadura tiene como gran ventaja el grado de seguridad del lenguaje con respecto al uso de las operaciones.

Otro mecanismo que pueden utilizar los lenguajes para la especificación de tipos es la *inferencia de tipos*. Este mecanismo es utilizado por algunos lenguajes funcionales en particular. Lo vamos a profundizar cuando discutamos este tipo de lenguajes, pero la idea es que el lenguaje tiene un algoritmo que le permite inferir los tipos vinculados a partir de las operaciones y los tipos mencionados de manera explícita. Así por ejemplo en las siguientes declaraciones de funciones:

```
Fun Mult10(x) = 10 * x  
Fun cuadrado(x):int = x * x
```

Se infiere que el tipo de la variable x es entero. En el primer caso porque resulta de la multiplicación con el entero 10, en el segundo porque el tipo resultado de la función es entero.

Por el otro lado, si la ligadura es **dinámica**, la asociación tiene lugar en general a través de las sentencias de asignación (por ejemplo, en PHP o JavaScript). Este tipo de ligadura tiene como gran ventaja el grado de flexibilidad; como contracara suele presentar altos costos (chequeo de tipos dinámico e interpretación). La detección de errores de tipos antes de la ejecución es muy difícil y a veces imposible.

Algunas consideraciones sobre el tiempo de vida

Mencionamos que está relacionado a la forma de administración de memoria. Con esto, hay variables que son:

- ✧ **Estáticas:** los objetos están asociados a direcciones absolutas de memoria. Esta asociación se retiene/mantiene durante toda la ejecución.
- ✧ **Pila:** se asignan o desasignan (allocate o deallocate) de manera LIFO, usualmente con las llamadas a rutinas y retornos.
- ✧ **Heap:** se asignan y desasignan en tiempos arbitrarios. Requiere algoritmos de manejo de almacenamiento más generales.

Mirá en el libro los cuatro tipos de variables que surgen de acuerdo a la forma de administración de memoria y pensá en las ventajas y desventajas de cada una: estáticas, dinámicas basadas en pila, dinámicas con heap explícito y dinámicas con heap implícito.

Algunas consideraciones sobre alcance y ambiente de referenciamiento

Los conceptos de **alcance** y **ambiente de referenciamiento** están relacionados pero no son lo mismo.

- ✧ El **alcance** de una entidad es la región de texto del programa en el cual una determinada entidad es visible (su ligadura está activa).
- ✧ El **ambiente de referenciamiento** es la colección o conjunto de entidades visibles (con ligaduras activas) en una determinada sentencia.

El **alcance** puede ser básicamente estático o dinámico. La forma y el tipo de reglas de alcance son fundamentales para la comprensión de los programas escritos en un lenguaje de programación. Si no se hace una buena lectura de cuáles son estas reglas de alcance, es muy probable que se comprometa seriamente el desarrollo correcto de programas en el lenguaje.

✧ **Alcance estático:** en un lenguaje con reglas de alcance estático o léxico, la ligadura se determina en tiempo de compilación a partir de la examinación del texto del programa.

Formas conocidas de reglas de alcance en lenguajes:

- a) Local – global.
- b) Bloques comunes e importación.
- c) Bloques anidados.
- d) Orden de declaración.

✧ **Alcance dinámico:** la ligadura se resuelve siguiendo la secuencia de llamados, no en la relación del texto del programa. De ahí que solo pueda determinarse en ejecución.

¿Qué sucede si el mapeo entre nombre y objetos no es uno-a-uno? La interpretación debería ser única ya que, de lo contrario, tendríamos problemas de ambigüedad. La única manera en la que esto podría suceder sería que haya:

- ✧ Aliasing: el mismo objeto de dato está ligado a más de un nombre en el alcance.
- ✧ Sobrecarga de operaciones.

El **ambiente de referenciamiento** de una sentencia es la colección de nombres que son visibles en la sentencia.

- ✧ En lenguajes con reglas de ambiente estático, el ambiente está conformado por todos los nombres locales más los nombres visibles de todos los ambientes que lo contienen.
- ✧ En lenguajes con reglas de ambiente dinámico, el ambiente está conformado nuevamente por todos los nombres locales, más los nombres visibles de todos los subprogramas activos. Recuerden que un subprograma está activo si su ejecución ha comenzado pero aún no ha finalizado.